SprechenAI

PROJE TESLİM RAPORU · V0.1

Gerçek hayat için Almanca konuşma koçu

Kurulan frontend, backend ve PostgreSQL hazırlığının; önemli dosyaların ve çalışan kullanıcı deneyiminin teknik açıklaması.

Next.js + TypeScript | Tailwind CSS | ASP.NET Core | PostgreSQL | AI entegrasyonuna hazır

HAZIRLANMA TARİHİ	ÇALIŞMA DİZİNİ	DURUM
4 Eylül 2026	SprechenAI	Build başarılı

NE OLUŞTURULDU?

İlk sürümün kapsamı

SprechenAI; günlük hayatta karşılaşılan durumlar üzerinden Almanca konuşma pratiği yaptıran, cümleyi yalnızca düzeltmekle kalmayıp **neden doğru olduğunu açıklayan** bir öğrenme ürünü olarak tasarlandı.

4
hazır frontend sayfası

10
gerçek hayat senaryosu

8
hazır API rotası

KULLANICI DENEYİMİ

Öğrenme akışı

Kullanıcı bir senaryo seçer, seviyesini ve açıklama derinliğini belirler, örnek konuşmaya katılır, gramer parçalarını inceler ve hatasını anında görür.

A1–B2 · Quick · Normal · Deep

TEKNİK TEMEL

Değiştirilebilir mimari

Frontend şimdilik yerel mock veriyle bağımsız çalışır. Backend tarafında yapay zekâ sağlayıcısı bir arayüz arkasındadır; ileride gerçek OpenAI istemcisi eklenebilir.

Mock bugün · OpenAI yarın

Tamamlanan başlıca özellikler

- ✓ Modern, responsive ana sayfa ve navigasyon
- ✓ Gerçekçi Almanca–Türkçe mock konuşma
- ✓ Masaüstü Grammar Inspector paneli
- ✓ Çalışan kullanıcı cevabı düzeltme akışı
- ✓ Controller + service + data katmanlı API
- ✓ PWA hazırlığı için web app manifest
- ✓ Aranabilir ve seçilebilir senaryo kartları
- ✓ Tıklanabilir NOM / AKK / DAT etiketleri
- ✓ Mobil grammar drawer deneyimi
- ✓ Mock gelişim yüzdeleri ve yaygın hatalar
- ✓ PostgreSQL için EF Core entity eşlemeleri
- ✓ Docker Compose veritabanı servisi

Bilerek yok

Bu aşamada gerçek OpenAI isteği, kimlik doğrulama ve kalıcı kullanıcı verisi uygulanmadı. Ekranlar ve API davranışları çalışan mock verilerle hazırlanmıştır.

BÜYÜK RESİM

Sistem nasıl bölündü?

Proje iki ana uygulamaya ayrılır. Bu ayrım arayüzün, API'nin ve veri katmanının birbirinden bağımsız geliştirilebilmesini sağlar.

Next.js UI
Sayfalar, bileşenler, responsive deneyim

→

ASP.NET API
Controller, sözleşme ve servisler

→

PostgreSQL
EF Core entity ve ilişkileri

```
SprechenAI/ ├ frontend/ Next.js uygulaması | ├ app/ Sayfalar, layout, global CSS | ├ components/ Tekrar kullanılabilir UI parçaları | ├ data/ Tipli mock içerikler | ├ types/ TypeScript veri tipleri | └ public/ PWA manifest ├ backend/ | ├ SprechenAI.slnx .NET solution | └ SprechenAI.Api/ | ├ Controllers/ HTTP giriş noktaları | ├ Services/ Uygulama iş kuralları | ├ Contracts/ Request/response modelleri | ├ Domain/Entities/ Veritabanı varlıkları | └ Data/ EF Core DbContext ├ docker-compose.yml PostgreSQL konteyneri ├ .env.example Ortam değişkeni örneği └ README.md Kurulum ve kullanım rehberi
```

Bugünkü çalışma şekli

Frontend doğrudan `data/` altındaki mock içerikleri kullanır. Böylece PostgreSQL veya backend kapalıyken bile bütün demo deneyimi çalışır.

Gelecekteki çalışma şekli

Mock veriler API servislerine taşınır; gerçek AI implementasyonu `IAiConversationClient` arayüzüne bağlanır ve sonuçlar EF Core üzerinden saklanır.

KULLANICI ARAYÜZÜ

Dört temel ekran

01 · HOME

Ana sayfa

Ürünün değer önerisini, çalışma yaklaşımını ve "Choose a scenario / Try the demo" aksiyonlarını sunar. Masaüstü ve mobil navigasyon buradan başlar.

`frontend/app/page.tsx`

02 · SCENARIOS

Senaryo seçimi

10 senaryoyu kart düzeninde gösterir. Arama alanı gerçekten filtreleme yapar; seçilen kart kullanıcıyı konuşma sayfasına götürür.

`frontend/app/scenarios/page.tsx`

03 · CONVERSATION

Konuşma

Almanca–Türkçe mesajlar, açıklama düğmesi, seviye/derinlik kontrolleri, gramer etiketleri ve cevap düzeltme akışını birleştirir.

`frontend/app/conversation/page.tsx`

04 · PROGRESS

Gelişim

Konu bazlı yüzdeler, pratik serisi, genel güven skoru ve "Common Mistakes" alanını gösterir. Zaman filtresi ve hata açıklaması etkileşimlidir.

`frontend/app/progress/page.tsx`

Senaryo kataloğu

Restaurant · Doctor · Work · Supermarket · Airport · Apartment Viewing · Meeting New People · Football · Date · Custom Scenario

Sayfa yönlendirme yapısı

URL	SAYFA	ANA KULLANICI AKSİYONU
`/`	Home	Ürünü tanı, senaryo veya demoya geç
`/scenarios`	Scenario Selection	Ara, kart seç, pratiği başlat
`/conversation`	Conversation	Mesajı incele, cevabı düzelt
`/progress`	Progress	Konuları ve tekrar eden hataları gör

Responsive Sayfalar mobil öncelikli Tailwind sınıflarıyla yazıldı. Üst navigasyon küçük ekranda alt menüye dönüşür; Grammar Inspector ise kalıcı yan panel yerine drawer olarak açılır.

DOSYA REHBERİ

Frontend'in kritik parçaları

DOSYA	GÖREVİ VE ÖNEMİ
`app/layout.tsx`	Uygulamanın kök layout'u. Metadata, viewport, manifest bağlantısı ve bütün sayfalarda kullanılan `AppShell` burada kurulur.
`app/globals.css`	Renk zemini, seçim/focus davranışı, animasyon, grid dokusu ve erişilebilir reduced-motion ayarları.
`components/layout/AppShell.tsx`	Header, içerik ve mobil alt navigasyonu tek bir ortak kabukta birleştirir.
`components/layout/Header.tsx`	Aktif rotayı algılayan masaüstü menüsü ve "Start practice" aksiyonu.
`components/layout/MobileNav.tsx`	Mobilde başparmak erişimine uygun, sabit alt navigasyon.
`components/scenarios/ScenarioGrid.tsx`	Arama, filtreleme ve seçili senaryo state'ini yönetir.
`components/conversation/ConversationWorkspace.tsx`	Konuşma sayfasının ana orkestratörü: seviye, açıklama detayı, seçili gramer ögesi ve mobil drawer state'i.
`components/conversation/ChatMessage.tsx`	Koç/kullanıcı mesajlarını, çeviriyi, etiketleri ve açılır cümle açıklamasını üretir.
`components/conversation/GrammarInspector.tsx`	Seçilen dil parçasının isim, artikel, hâl, sebep, dönüşüm, kural, örnek ve anlam ayrıntılarını gösterir.
`components/conversation/CorrectionComposer.tsx`	Kullanıcının örnek cevabını gerçekten kontrol eder; yanlış/doğru varyant ve Dativ sebebini gösterir.
`data/conversation.ts`	Konuşma mesajlarıyla yapılandırılmış gramer açıklamalarının tek, tipli mock veri kaynağı.
`types/index.ts`	`LearningLevel`, `GrammarItem`, `ConversationMessage` gibi ortak TypeScript sözleşmeleri.
`tailwind.config.ts`	Marka renkleri, gölgeler, yuvarlaklıklar ve taranacak kaynak yolları.
`public/manifest.webmanifest`	Ad, kısa ad, tema rengi ve standalone davranışıyla ileride PWA'ya dönüşümün başlangıcı.

Neden parçalara ayrıldı? State yönetimi, görsel sunum ve mock veri aynı dosyada toplanmadı. Böylece gerçek API bağlandığında UI bileşenlerinin yeniden yazılması gerekmez.

TEMEL ÜRÜN ÖZELLİĞİ

Cümle neden böyle kurulmuş?

ÖRNEK ALMANCA CÜMLE

Ich gehe mit meinem Freund ins Kino.

Arkadaşımla sinemaya gidiyorum.

NOM · Ich | VERB · gehe | DAT · mit meinem Freund | AKK · ins Kino

mit meinem Freund

Edat	**mit** her zaman Dativ alır.
Temel isim	Freund = **der Freund**
Dönüşüm	der Freund → **dem Freund**
Possessiv	mein Freund → **meinem Freund**

ins Kino

Temel isim	Kino = **das Kino**
Kısaltma	in + das = **ins**
Anlam	Bir hedefe doğru hareket
Sonuç	“Wohin?” → **Akkusativ**

Deep görünümdeki artikel pusulası

ARTİKEL TÜRÜ	NOMİNATİV	AKKUSATİV	DATİV
Belirli artikel	der	den	dem
Belirsiz artikel	ein	einen	einem
İyelik artikeli	mein	meinen	meinem

Kısaltma örnekleri

in + das = ins
in + dem = im
an + das = ans
zu + dem = zum

Açıklama derinliği

Quick: sebep + basit örnek
Normal: dönüşüm ve edat
Deep: kural + kelime sırası + referans

ÇALIŞAN MOCK ETKİLEŞİMLER

Hata, açıklamaya dönüşüyor

Kullanıcı cevabı yalnızca kırmızı işaretlenmez. Yanlış parça, doğru karşılığı ve kural aynı anda gösterilir.

YANLIŞ

~~mit meinen Freund~~

DOĞRU

mit meinem Freund

Açıklama **mit** Dativ gerektirir. Freund = **der Freund**. Eril Dativ'te iyelik artikeli **mein** → **meinem** olur.

Kullanıcı yazar
Textarea state güncellenir

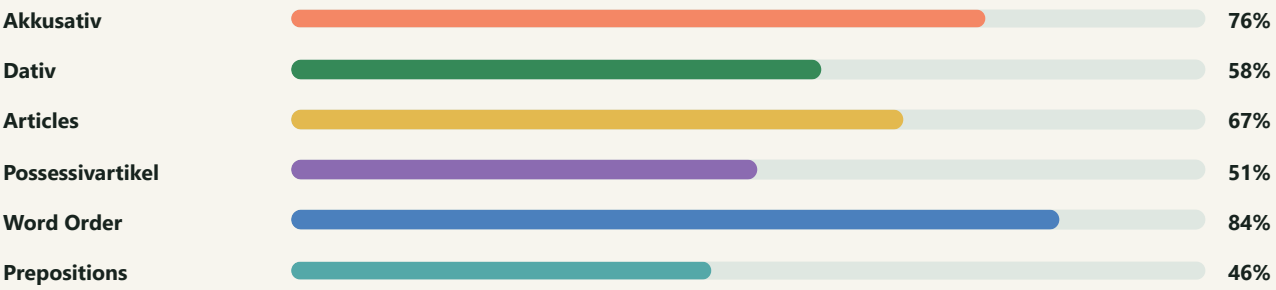
→

Form kontrol eder
Noktalama ve büyük/küçük harf normalize edilir

→

Sonuç görünür
Doğru veya açıklamalı düzeltme

Progress sayfasındaki örnek ölçümler



Common Mistakes

Yanlış/doğru metin, konu etiketi ve tekrar sayısı birlikte sunulur. Örnekte Dativ hatası **4 kez** görülmüş kabul edilir.

Pratiğe geri dönüş

"Practice this pattern" bağlantısı kullanıcıyı doğrudan konuşma ekranına taşır. Hata açıklaması açılıp kapanabilir.

EKRAN UYUMU

Masaüstü ve mobil davranışı

MASAÜSTÜ

Kalıcı Grammar Inspector

Konuşma sağ taraftaki yaklaşık 350 px'lik inspector ile iki sütunlu görünür. Panel scroll sırasında üstte sabit kalır; açıklama her zaman erişilebilirdir.

MOBİL

Drawer + alt navigasyon

Inspector normal akışı daraltmaz. “Inspect grammar” düğmesiyle alttan açılan, kaydırılabilir bir drawer olur. Navigasyon da ekranın altına taşınır.

Responsive tasarım kararları

- ✓ Mobil öncelikli Tailwind breakpoint'leri
- ✓ Dar ekranda tek sütuna düşen kartlar
- ✓ Focus-visible erişilebilirlik işaretleri
- ✓ Sticky masaüstü inspector
- ✓ Dokunma alanı büyük aksiyon düğmeleri
- ✓ Esnek konuşma balonu genişlikleri
- ✓ Reduced-motion tercihine uyum
- ✓ Modal semantiğine sahip mobil drawer

PWA'ya hazırlanan parçalar

PARÇA	ŞU AN NE SAĞLIYOR?	SONRAKİ ADIM
`manifest.webmanifest`	Uygulama adı, kısa adı, tema, arka plan, başlangıç URL'si ve standalone display.	192/512 px ikonlar
`layout.tsx`	Manifest bağlantısı ve mobil viewport/theme metadata.	Apple metadata ve ikon
Responsive shell	Telefonda native uygulamaya yakın menü ve tam ekran akış.	Install prompt UX
Yerel mock veri	API kapalıyken demo kullanılabilir.	Service worker cache stratejisi

Not Bu sürüm “PWA olmaya hazır” altyapıyı kurar; henüz service worker veya offline senkronizasyon eklemez. Böylece ilk adım gereksiz karmaşıklık olmadan tamamlanmıştır.

ASP.NET CORE WEB API

İstekler nasıl işleniyor?

Controller
HTTP route ve durum kodu

→

Service
Uygulama davranışı

→

AI / Data sınırı
Mock bugün, gerçek servis sonra

METHOD	ROUTE	DAVRANIŞ
GET	`/health`	Veritabanına ihtiyaç duymadan API liveness cevabı verir.
GET	`/health/database`	PostgreSQL bağlantısını ayrıca doğrular; yoksa 503 verir.
POST	`/api/conversations`	Senaryo, seviye ve detayla mock konuşma başlatır; 201 döner.
GET	`/api/conversations/{id}`	Bellekte tutulan konuşmayı kimliğiyle getirir.
POST	`/api/messages`	Mock koç cevabı ve gerekirse düzeltme döndürür.
POST	`/api/grammar/analyze`	Seçili ifade için yapılandırılmış gramer analizi verir.
GET	`/api/progress`	Altı konu için mock ilerleme yüzdelerini döndürür.
GET	`/api/mistakes`	Tekrarlanan hata, doğru şekil, konu ve sayıyı döndürür.

Mock AI sınırı

```
public interface IAiConversationClient
{
    Task<MessageResponse> ReplyAsync(...);
    Task<GrammarAnalysisResponse> AnalyzeAsync(...);
}
```

`MockAiConversationClient` bu arayüzü bugün sabit, tahmin edilebilir cevaplarla uygular. İleride `OpenAiConversationClient` aynı arayüzü uygular; controller veya conversation service değişmeden kalabilir.

Temiz sınır API sözleşmeleri entity'lerden ayrıdır. Dış dünyaya doğrudan veritabanı nesnesi göndermek yerine `Contracts/` altındaki request/response record'ları kullanılır.

DOSYA REHBERİ

Backend'in kritik parçaları

DOSYA	GÖREVİ VE ÖNEMİ
`Program.cs`	DbContext, controller, CORS, problem details ve servis dependency injection kayıtlarının merkezi. Development'ta HTTP, production'da HTTPS yönlendirmesi uygular.
`appsettings.json`	PostgreSQL connection string, frontend origin ve log seviyelerini barındırır.
`Data/SprechenAiDbContext.cs`	Altı DbSet'i ve User–Conversation–Message–Analysis gibi ilişkileri Fluent API ile tanımlar.
`Services/IAiConversationClient.cs`	Gelecekteki gerçek AI sağlayıcısının uyması gereken iki operasyonluk sınır.
`Services/MockAiConversationClient.cs`	AI çağrısı yapmadan mesaj cevabı, Dativ düzeltmesi ve gramer analizi üretir.
`Services/ConversationService.cs`	Konuşmayı başlatır ve thread-safe `ConcurrentDictionary` içinde geçici olarak tutar.
`Services/ProgressService.cs`	Progress ve mistake controller'larının kullandığı mock öğrenme verisini sağlar.
`Controllers/HealthController.cs`	API ve PostgreSQL sağlığını birbirinden ayırır; DB kapalıyken ana health çalışır.
`Controllers/GrammarController.cs`	`POST /api/grammar/analyze` isteğini AI client sınırına yönlendirir.
`Contracts/*Contracts.cs`	JSON giriş/çıkış biçimlerini güçlü tipli C# record'larıyla tanımlar.
`Domain/Entities/*.cs`	Gelecekte PostgreSQL'de saklanacak alanları ve navigation property'leri içerir.
`SprechenAI.Api.csproj`	.NET 9 hedefini, EF Core Design ve Npgsql PostgreSQL paketlerini tanımlar.

CORS

Varsayılan olarak `http://localhost:3000` frontend origin'ine izin verir. Değer konfigürasyondan değiştirilebilir.

Hata formatı

`AddProblemDetails()` beklenmeyen API hataları için standartlaştırılmış cevap altyapısı sağlar.

VERİ KATMANI

Hazırlanan entity'ler

User

DisplayName
Email (unique)
LearningLevel

1 → N

Conversation

Scenario
LearningLevel
ExplanationDetail
CompletedAt

1 → N

Message

Role
GermanText
TurkishTranslation

GrammarAnalysis

Message ile 1:1
SelectedPhrase
BaseNoun / Article
Case / Explanation
Transformation / Labels

Mistake

User ile N:1
Incorrect / Correct
Topic / Explanation
OccurrenceCount

UserProgress

User ile N:1
Topic
MasteryPercentage
Attempts
LastPracticedAt

Önemli veri tasarımı kararları

Ortak kimlik ve zaman

Bütün entity'ler `BaseEntity` üzerinden Guid `Id`, `CreatedAt` ve `UpdatedAt` alır.

Kontrollü cascade

User silinirse bağlı konuşma, hata ve ilerleme kayıtları; conversation silinirse mesajları; message silinirse analizi silinecek biçimde eşlendi.

Tekillik

Email unique'tir. Ayrıca aynı kullanıcı + konu için birden fazla `UserProgress` kaydı oluşması engellenir.

Henüz migration yok

Entity ve mapping'ler hazırdır; ilk migration kalıcı veri geliştirmesi başladığında kontrollü olarak üretilecektir.

Docker Compose PostgreSQL

```
docker compose up -d postgres

Database : sprechenai
Username : sprechenai
Port     : 5432
Image    : postgres:16-alpine
```

Kalıcı Docker volume ve `pg_isready` healthcheck tanımlıdır. Değerler kökteki `.env` ile değiştirilebilir; örnek değerler `.env.example` içindedir.

GELİŞTİRİCİ TESLİMİ

Projeyi çalıştırma

Frontend

```
cd frontend
npm install
npm run dev
```

Tarayıcı: **http://localhost:3000**

Backend

```
cd backend
dotnet restore SprechenAI.slnx
dotnet run --project \
  SprechenAI.Api/SprechenAI.Api.csproj \
  --launch-profile http
```

API: **http://localhost:5080**

Gerçekleştirilen kontroller

KONTROL	SONUÇ	NE DOĞRULANDI?
`npm run typecheck`	**Başarılı**	Strict TypeScript tip kontrolü
`npm run build`	**Başarılı**	4 sayfanın optimize production build'i
`dotnet build`	**Başarılı**	0 hata, 0 uyarı
`GET /health`	**200**	API çalışma zamanı
`POST /api/conversations`	**201**	Konuşma oluşturma cevabı
`POST /api/grammar/analyze`	**200**	Yapılandırılmış Dativ açıklaması
Headless Chrome kontrolü	**Başarılı**	Mobil conversation görünümü

Bu bilgisayarda hazır

Node.js 24 LTS
.NET SDK 9+
npm bağımlılıkları ve lockfile

Eksik prerequisite

Docker Desktop veya yerel PostgreSQL 16
Bu nedenle DB health endpoint'i container başlatıldıktan sonra kontrol edilmelidir.

Ana rehber Tüm komutlar, ortam değişkenleri, endpoint listesi ve migration akışı proje kökündeki `README.md` dosyasında da bulunur.

TESLİM ÖZETİ

Sağlam bir ilk ürün temeli

SprechenAI'nin ilk adımı yalnızca statik ekranlardan oluşmuyor. Kullanıcı açısından bütün ana etkileşimler çalışıyor; teknik tarafta ise gerçek AI ve kalıcı veriye geçiş için açık sınırlar hazır.

ŞİMDİ

Demo edilebilir

Senaryo seçimi, konuşma, gramer analizi ve progress akışı uçtan uca gösterilebilir.

SONRA

API'ye bağlanabilir

Frontend mock kaynakları servis çağrılarıyla değiştirilebilir; component yapısı korunur.

DAHA SONRA

AI eklenebilir

`IAiConversationClient` için gerçek OpenAI implementasyonu yazılabilir.

Önerilen sonraki geliştirme sırası

01 · Migration	İlk EF Core migration'ını üret, development veritabanını oluştur.
02 · API client	Frontend için tipli bir fetch/service katmanı ekle ve mock/API modu seçilebilir yap.
03 · Kimlik	User entity'sini gerçek oturum ve kullanıcı profiliyle bağla.
04 · OpenAI	Prompt, structured output, güvenlik ve hata yönetimiyle gerçek client'ı uygula.
05 · Persistence	Conversation, Message, Mistake ve UserProgress verilerini kalıcılaştır.
06 · PWA	İkon, service worker, cache ve offline/install deneyimini tamamla.
07 · Test	Frontend component testleri, API integration testleri ve E2E akışları ekle.

TEK CÜMLEYLE

SprechenAI artık modern, responsive, çalışan mock deneyime ve büyümeye uygun teknik omurgaya sahip.

Rapor kaynağı: docs/SprechenAI_Proje_Raporu.html